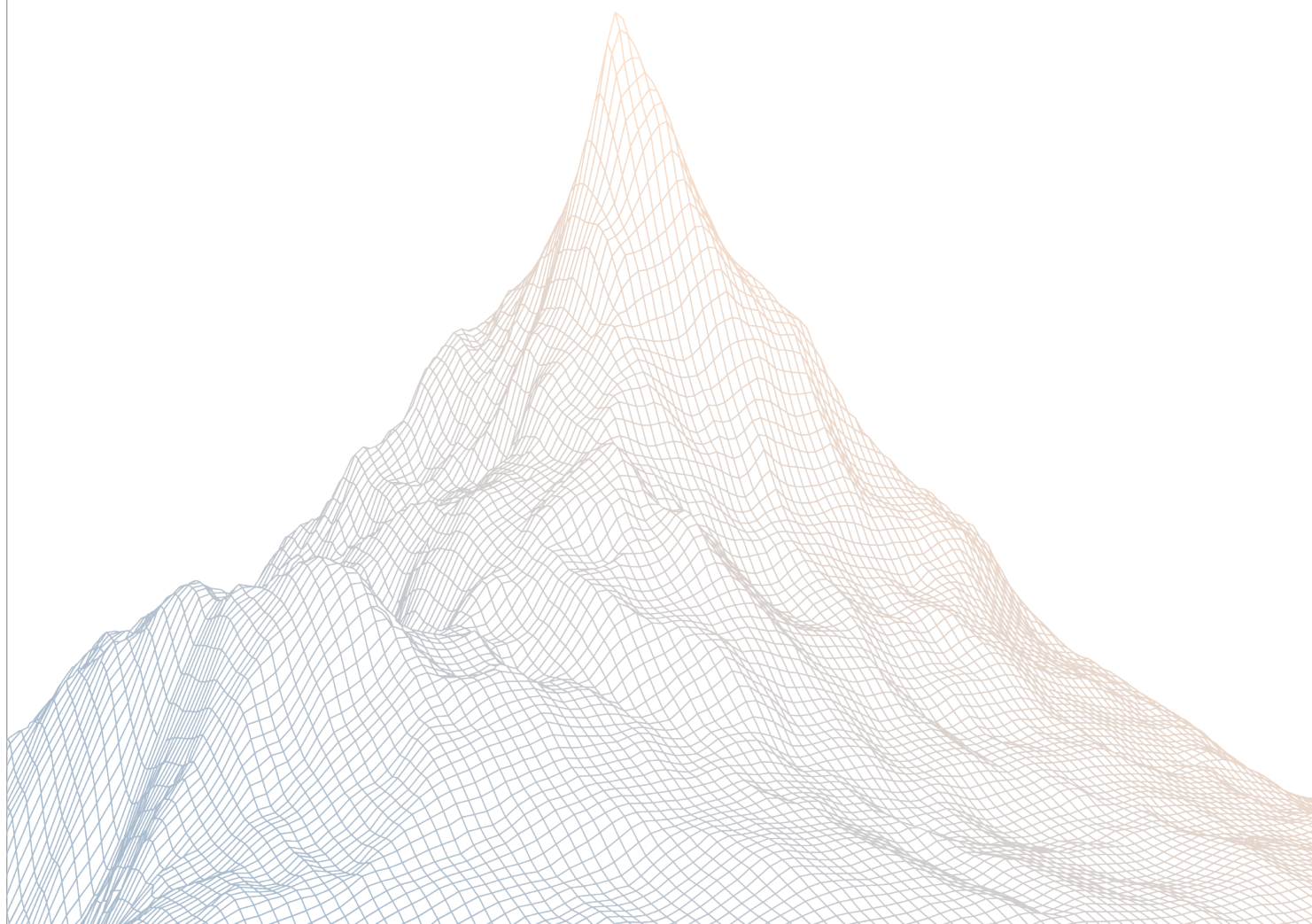


Meteora

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

May 20th to May 28th, 2026

AUDITED BY:

J4X
peakbolt

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Meteora	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	6
4	Findings	7
4.1	Low Risk	8
4.2	Informational	15

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	dynamic-bonding-curve
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve/pull/193
Commit Hash	5c9b0b14b54446357dc11467f7ab248ba6c0f62b
Files	Diff from b4f954733f0e88258f1eb3f0eff75e4314c9610c
Target	dynamic-bonding-curve
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve/pull/196
Commit Hash	5c3ad521c8847984bc1ed739c37e1f6a50be2610
Files	Diff from 00f28a30fe8a7598c9b0dd58c19732e29c7f1b96

Target	dynamic-bonding-curve
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve/pull/198
Commit Hash	3114aeb355087abeb217a1e64a347e6f5466b616
Files	Diff from c96948db59c4e450878c6eb7e7af3533bfc7b4b6
Target	PR-193 Mitigation Review
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve/pull/193
Commit Hash	15e98f293059d0b14f2e8bf86a334c3454b602c2
Files	Diff from b4f954733f0e88258f1eb3f0eff75e4314c9610c
Target	PR-196 Mitigation Review
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve/pull/196
Commit Hash	15e98f293059d0b14f2e8bf86a334c3454b602c2
Files	Diff from 00f28a30fe8a7598c9b0dd58c19732e29c7f1b96
Target	PR-198 Mitigation Review
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve/pull/198
Commit Hash	15e98f293059d0b14f2e8bf86a334c3454b602c2
Files	Diff from c96948db59c4e450878c6eb7e7af3533bfc7b4b6

2.3 Audit Timeline

May 20, 2026	Audit start
May 28, 2026	Audit end
June 1, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	3
Informational	9
Total Issues	12

3

Findings Summary

ID	Description	Status
L-1	Creator quote-fee claims can be blocked by a malicious zero-amount base hook	Resolved
L-2	Base protocol fees can remain unclaimable for incomplete transfer-hook pools	Acknowledged
L-3	Fee claim and swap handlers hold pool borrows across transfer-hook CPIs	Resolved
I-1	Protocol fee claim event in <code>claim_protocol_fee2</code> uses truncatable <code>emit!()</code>	Acknowledged
I-2	SDK quote helpers require <code>VirtualPool</code> instead of <code>PoolState</code>	Resolved
I-3	Protocol surplus is not capped by the maximum quote claim amount	Resolved
I-4	Transfer-hook pool metadata is not distinguishable from ordinary pool metadata	Acknowledged
I-5	<code>QuoteToBase</code> trade direction documentation is incorrect	Resolved
I-6	Mint-authority comments misstate transfer-hook support	Resolved
I-7	Config loader panics on short config account data	Resolved
I-8	Transfer-hook handlers accept unconsumed or unused remaining accounts	Resolved
I-9	README omits the transfer-hook-only restriction for mint-authority options	Resolved

4

Findings

4.1 Low Risk

A total of 3 low risk findings were identified.

[L-1] Creator quote-fee claims can be blocked by a malicious zero-amount base hook

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [ix_claim_creator_trading_fee.rs#92](#)

Description:

`claim_creator_trading_fee` transfers accrued creator fees from the pool vaults to the creator's receiver token accounts.

However, it calls `transfer_token_from_pool_authority` for the base token before transferring quote tokens even when `token_base_amount` is zero. That is because the transfer hook are always triggered regardless of amount.

A malicious base token hook can reject the zero-amount base transfer and prevent the creator from claiming their nonzero quote token fees.

Example Scenario:

A creator has `token_base_amount = 0` and `token_quote_amount = 1_000` from accrued trading fees. The base mint has a malicious transfer hook that rejects the zero-amount `transfer_token_from_pool_authority()` call, so the instruction reverts before the quote transfer executes.

```
transfer_token_from_pool_authority(  
  ctx.accounts.pool_authority.to_account_info(),  
  &ctx.accounts.base_mint,  
  &ctx.accounts.base_vault,  
  ctx.accounts.token_a_account.to_account_info(),  
  &ctx.accounts.token_base_program,
```



```
        token_base_amount,  
        parsed_transfer_hook_accounts.transfer_hook_base,  
    )?;  
  
    transfer_token_from_pool_authority(  
        ctx.accounts.pool_authority.to_account_info(),  
        &ctx.accounts.quote_mint,  
        &ctx.accounts.quote_vault,  
        ctx.accounts.token_b_account.to_account_info(),  
        &ctx.accounts.token_quote_program,  
        token_quote_amount,  
        None,  
    )?;
```

Recommendations:

Skip the base transfer_token_from_pool_authority call when token_base_amount is zero, and skip any other token transfer whose computed amount is zero.

```
if token_base_amount > 0 {  
    transfer_token_from_pool_authority(  
        ctx.accounts.pool_authority.to_account_info(),  
        &ctx.accounts.base_mint,  
        &ctx.accounts.base_vault,  
        ctx.accounts.token_a_account.to_account_info(),  
        &ctx.accounts.token_base_program,  
        token_base_amount,  
        parsed_transfer_hook_accounts.transfer_hook_base,  
    )?;  
}
```

Meteora: Resolved with [@199188dd14 ...](#) and [@efb3414692 ...](#)

Zenith: Verified.

[L-2] Base protocol fees can remain unclaimable for incomplete transfer-hook pools

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [src/instructions/operator/ix_claim_protocol_fee2.rs#L116-L128](#)
- [src/state/fee.rs#L120-L143](#)
- [src/state/virtual_pool.rs#L940-L977](#)

Description:

Transfer-hook pools may be configured with `CollectFeeMode::OutputToken`. In that mode, a `QuoteToBase` swap collects fees in the base token: `FeeMode::get_fee_mode` sets `fees_on_base_token` to true, and `PoolState::apply_swap_result` records the protocol share in `protocol_base_fee`.

The new protocol fee claim path does not allow those base-denominated fees to be collected from an incomplete transfer-hook pool. When `claim_protocol_fee2` identifies a base-token claim, it requires a transfer-hook pool to have already completed its curve before calling `claim_protocol_base_fee`:

```
let amount = if is_claiming_base {
    if pool_loader.is_transfer_hook_pool() {
        require!(
            pool.is_curve_complete(config.migration_quote_threshold),
            PoolError::PoolIsIncompleted
        );
    }
    pool.claim_protocol_base_fee(max_amount)?
} else {
    pool.claim_protocol_quote_fee_and_surplus(max_amount,
        config.migration_quote_threshold)?
};
```

As a result, `QuoteToBase` swaps on an incomplete transfer-hook pool using `OutputToken` fee collection can continue to accrue protocol base fees while the only protocol fee claim path for those fees rejects the claim. If the curve is abandoned and never completes, the

accrued base-token protocol fees remain recorded in the pool but cannot be collected by the protocol. Quote-denominated protocol fees are not affected by this completion check.

Recommendations:

We recommend preventing collection of fees in base token for transfer hook pools, or allowing a way for the protocol to still claim fees in case of abandonment (dead man switch after 1 year).

Meteora: Acknowledged.

[L-3] Fee claim and swap handlers hold pool borrows across transfer-hook CPIs

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [ix_claim_partner_trading_fee.rs#L81](#)
- [ix_claim_creator_trading_fee.rs#L70](#)
- [process_swap.rs#L114](#)
- [process_swap.rs#L298](#)
- [process_swap.rs#L311](#)
- [process_swap.rs#L330](#)

Description:

The partner and creator trading-fee claim handlers debit fee accounting before transferring tokens from the pool vaults.

However, they keep RefMut pool borrows, and the partner path also keeps a config borrow, alive across token transfers that can invoke transfer-hook CPIs.

The same pattern also appears in `swap2_with_transfer_hook`. `process_swap` holds `pool_loader.load_mut()` and `config_loader.load()` borrows while invoking transfer-hook-capable token transfers:

- `transfer_token_from_user` for the input transfer
- `transfer_token_from_pool_authority` for the output transfer
- `transfer_token_from_pool_authority` for the base referral transfer, when applicable

Transfer hooks whose extra accounts include the DBC pool or config can fail with runtime borrow errors, making valid hook integrations incompatible with fee claiming or transfer-hook swaps.

Example Scenario:

1. A transfer-hook pool accrues 100 base tokens of partner fees.
2. The hook's `ExtraAccountMetaList` requires the DBC pool account so the hook can inspect pool state.

3. The partner calls `claim_trading_fee2` with the required hook accounts.
4. DBC still holds `pool_loader.load_mut()` while invoking the token transfer.
5. The hook tries to read the pool account and fails due to the outstanding mutable borrow.

Additional Swap Scenario:

1. A user calls `swap2_with_transfer_hook` on a transfer-hook pool.
2. `process_swap` loads the pool mutably and applies the swap result.
3. The base token hook's `ExtraAccountMetaList` requires the DBC pool account so the hook can inspect pool state.
4. DBC still holds the mutable pool borrow while invoking the input, output, or referral transfer.
5. The hook tries to read the pool account and fails due to the outstanding mutable borrow.

```
let mut pool = pool_loader.load_mut()?;
let (token_base_amount, token_quote_amount) =
    pool.claim_partner_trading_fee(max_base_amount, max_quote_amount)?;

transfer_token_from_pool_authority( ... )?;
```

```
let mut pool = pool_loader.load_mut()?;
// ...
pool.apply_swap_result( ... )?;

transfer_token_from_user( ... )?;
transfer_token_from_pool_authority( ... )?;
```

Recommendations:

Scope the state mutation so pool and config borrows are dropped before external token or hook CPIs.

```
let (token_base_amount, token_quote_amount) = {
    let mut pool = pool_loader.load_mut()?;
    pool.claim_partner_trading_fee(max_base_amount, max_quote_amount)?
};
```

For `process_swap`, compute and persist the swap state changes first, copy out the values needed for transfers/events, then drop the pool/config borrows before invoking transfer-hook token CPIs. If post-transfer state updates are still required, reacquire the pool borrow after the external transfers.

Meteora: Resolved with [@681cf3f5f7 ...](#)

Zenith: Verified.

4.2 Informational

A total of 9 informational findings were identified.

[I-1] Protocol fee claim event in `claim_protocol_fee2` uses truncatable `emit!()`

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [ix_claim_protocol_fee2.rs#L159](#)

Description:

Consider using `emit_cpi!` for `claim_protocol_fee2` instead of `emit!` to prevent truncation of event log.

```
#[event_cpi]
#[derive(Accounts)]
pub struct ClaimProtocolFee2Ctx<'info> {
    /// receiver token account for the claimed token. validated through the
    /// protocol_fee program
    #[account(mut)]
    pub receiver_token_account: Box<InterfaceAccount<'info, TokenAccount>>,

    // emit! log could be truncated. should not rely on this
    emit!(EvtClaimProtocolFee2 {
    emit_cpi!(EvtClaimProtocolFee2 {
        // no transfer hook event variant, since this internal operation
        pool: ctx.accounts.pool.key(),
        receiver_token_account: ctx.accounts.receiver_token_account.key(),
        token_mint: token_mint.key(),
        amount,
    });
}
```

Meteora: Acknowledged. This is an intentional trade off to save 1 account as we don't

need to pass event_authority account.

[I-2] SDK quote helpers require VirtualPool instead of PoolState

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [src/quote_exact_in.rs#L5-L10](#)
- [src/quote_exact_out.rs#L5-L10](#)
- [src/quote_partial_fill.rs#L5-L10](#)

Description:

PR #193 split the pool account representation into a shared internal `PoolState` and account wrappers. The on-chain state now defines `PoolState` as the common data structure, while both `VirtualPool` and `TransferHookPool` wrap it in a `pool_state` field and dereference to it. The generic on-chain `PoolAccountLoader` follows this model: it accepts either the `VirtualPool` or `TransferHookPool` discriminator, then returns a borrowed `PoolState` for shared logic.

The Rust SDK quote helpers did not make the same abstraction update. `quote_exact_in`, `quote_exact_out`, and `quote_partial_fill` still import `VirtualPool` and require `pool: &VirtualPool`. Each helper only calls shared state methods such as `is_curve_complete`, `get_swap_result_from_exact_input`, `get_swap_result_from_exact_output`, and `get_swap_result_from_partial_input`, all of which are available through the underlying `PoolState` implementation. None of the quote paths require the outer `VirtualPool` account discriminator or any field that is specific to the non-transfer-hook wrapper.

The account split is implemented by making both wrappers contain the same internal state:

```
pub struct VirtualPool {
    pub pool_state: PoolState,
}

pub struct TransferHookPool {
    pub pool_state: PoolState,
}
```

The on-chain generic loader then accepts either wrapper discriminator and returns the shared state:

```
let is_transfer_hook = disc == TransferHookPool::DISCRIMINATOR;

require!(
    disc == VirtualPool::DISCRIMINATOR || is_transfer_hook,
    PoolError::InvalidPoolAccount
);

pub fn load(&self) → Result<Ref<'_, PoolState>> { ... }
```

In contrast, the SDK quote helpers still require the non-transfer-hook wrapper:

```
use dynamic_bonding_curve::state::{fee::FeeMode, PoolConfig, SwapResult2,
    VirtualPool};

pub fn quote_exact_in(
    pool: &VirtualPool,
    config: &PoolConfig,
    ...
) → Result<SwapResult2> {
```

As a result, SDK consumers that decode a transfer-hook pool as `TransferHookPool` cannot pass its internal `pool_state` into the quote API without wrapping, copying, or reinterpreting the account as the wrong wrapper type. This is inconsistent with the on-chain design, where shared pool logic is intentionally performed on `PoolState` after accepting both pool account wrappers. The affected SDK API therefore excludes the newly introduced transfer-hook pool type from otherwise wrapper-agnostic quote calculations.

A minimal reproduction is to decode a transfer-hook pool account with the generated Anchor coder and try to call any SDK quote helper with `decoded.pool_state`. The call does not type-check because the helper requires `&VirtualPool`, even though the quote math only reads `PoolState` fields and methods.

Recommendations:

We recommend changing the SDK quote helper signatures and tests to accept `&PoolState` instead of `&VirtualPool`, and decoding either account wrapper to its internal `pool_state` before quote calculation.

Meteora: Resolved with [@31d681077e...](#)

Zenith: Verified.

[I-3] Protocol surplus is not capped by the maximum quote claim amount

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [src/state/virtual_pool.rs#L1071-L1086](#)
- [src/instructions/operator/ix_claim_protocol_fee2.rs#L116-L128](#)

Description:

`claim_protocol_fee2` accepts a `max_amount` parameter and forwards it to the quote-token claim path when the receiver token account is for the quote mint:

```
let amount = if is_claiming_base {  
    /* base claim path */  
    pool.claim_protocol_base_fee(max_amount)?  
} else {  
    pool.claim_protocol_quote_fee_and_surplus(max_amount,  
        config.migration_quote_threshold)?  
};
```

The quote helper applies `max_amount` only to the tracked protocol quote fee buckets. `claim_protocol_quote_fee` first caps the amount taken from `protocol_quote_fee`, then caps the amount taken from `protocol_migration_quote_fee_amount` using the remaining limit. After that capped fee claim returns, `claim_protocol_quote_fee_and_surplus` adds the full protocol surplus when the curve is complete and protocol surplus has not already been withdrawn:

```
let mut amount = self.claim_protocol_quote_fee(max_amount)?;  
  
if self.is_protocol_withdraw_surplus == 0  
    && self.is_curve_complete(migration_quote_threshold)  
{  
    self.update_protocol_withdraw_surplus();  
    let protocol_surplus_amount =  
        self.get_protocol_surplus(migration_quote_threshold)?;  
    amount = amount.safe_add(protocol_surplus_amount)?;
```

```
}
```

As a result, the total quote-token transfer made by `claim_protocol_fee2` can exceed the caller-supplied `max_amount`. For example, after curve completion, a quote claim with `max_amount = 0` can still withdraw the full protocol surplus if `is_protocol_withdraw_surplus = 0`.

This is not an unauthorized withdrawal path. The instruction is gated by the protocol-fee authority signer, the surplus is the protocol surplus share, and the path does not appear to double-count partner or creator surplus. Migration accounting also remains separate from the claimed surplus amount.

The issue is therefore limited to parameter semantics and integration safety. If `max_amount` is intended to mean "maximum protocol fee bucket amount," then the current behavior is consistent with that narrower meaning but should be documented on the `claim_protocol_fee2` interface. If callers or wrapper programs reasonably treat `max_amount` as a hard maximum quote-token transfer amount, they can receive a larger transfer than requested once protocol surplus is available.

Recommendations:

We recommend making the intended API contract explicit. If `max_amount` should be a hard cap on the quote-token transfer, apply it to the total amount returned by `claim_protocol_quote_fee_and_surplus`, including protocol surplus. If it is only meant to cap protocol fee buckets, rename or document the parameter so integrators know that protocol surplus can be added on top of the requested maximum.

Meteora: Resolved with [@029d8e57de ...](#)

Zenith: Verified.

[I-4] Transfer-hook pool metadata is not distinguishable from ordinary pool metadata

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [src/instructions/creator/ix_create_virtual_pool_metadata.rs#L18-L56](#)
- [src/event.rs#L20-L24](#)

Description:

PR 193 introduces TransferHookPool as a distinct pool account type and separates several indexer-facing flows with transfer-hook-specific events. For example, config creation, pool initialization, swap, and curve-completion events each expose a transfer-hook variant in [event.rs](#).

The metadata creation path does not preserve that distinction. [create_virtual_pool_metadata](#) is now exposed for both VirtualPool and TransferHookPool. Its account context accepts the pool as an UncheckedAccount, and the access-control path accepts either pool discriminator before metadata is created. The handler then stores the pool key in VirtualPoolMetadata and emits the ordinary EvtVirtualPoolMetadata event:

```
pub virtual_pool: UncheckedAccount<'info>,

emit_cpi!(EvtVirtualPoolMetadata {
    virtual_pool_metadata: ctx.accounts.virtual_pool_metadata.key(),
    virtual_pool: ctx.accounts.virtual_pool.key(),
});
```

EvtVirtualPoolMetadata only contains the metadata account and referenced pool key. The [VirtualPoolMetadata](#) account also only stores that pool key plus display metadata. Neither surface indicates whether the referenced pool is the ordinary pool type or the untrusted transfer-hook pool type.

As a result, metadata consumers cannot preserve the transfer-hook isolation boundary from the metadata event or metadata account alone. A frontend or indexer that uses this metadata surface for pool discovery, labeling, or enrichment must perform an additional pool-account discriminator check to avoid presenting transfer-hook pool metadata through

an ordinary-pool pipeline.

Recommendations:

We recommend preserving the pool subtype on the metadata surface, for example, by adding a transfer-hook-specific metadata event.

Meteora: Acknowledged.

[I-5] QuoteToBase trade direction documentation is incorrect

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [src/params/swap.rs#L6-L10](#)

Description:

The TradeDirection enum documents both swap directions as Input base token, output quote token. This is correct for BaseToQuote, but the same comment is also attached to QuoteToBase.

```
/// Input base token, output quote token
BaseToQuote,
/// Input base token, output quote token
QuoteToBase,
```

The runtime swap path treats QuoteToBase as the opposite direction. In [process_swap](#), non-base input selects TradeDirection::QuoteToBase, and that branch uses quote_mint and quote_vault as the input side while using base_mint and base_vault as the output side.

The incorrect enum documentation can mislead maintainers and integrators reading the shared trade-direction type when reasoning about fee handling, swap math, or event semantics for quote-to-base swaps.

Recommendations:

We recommend updating the QuoteToBase documentation to state that it uses quote-token input and base-token output.

Meteora: Resolved with [@a7649a8915 ...](#)

Zenith: Verified.

[I-6] Mint-authority comments misstate transfer-hook support

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [src/state/config.rs#L379-L382](#)

Description:

The comments above `TokenAuthorityOption::CreatorUpdateAndMintAuthority` and `TokenAuthorityOption::PartnerUpdateAndMintAuthority` state that creating a config or initializing a pool with either option will fail:

```
// Deprecated. Creating a config or initializing a pool with this option
// will fail
CreatorUpdateAndMintAuthority,
// Deprecated. Creating a config or initializing a pool with this option
// will fail
PartnerUpdateAndMintAuthority,
```

That description is only true for the non-transfer-hook paths.

`ConfigParameters::validate` rejects mint-authority variants only when `is_transfer_hook` is false:

```
let token_authority_option =
    TokenAuthorityOption::try_from(self.token_update_authority)
        .map_err(|_| PoolError::InvalidTokenAuthorityOption)?;
// mint authority variants are only allowed for transfer-hook configs
require!(
    is_transfer_hook || !token_authority_option.has_mint_authority(),
    PoolError::InvalidTokenAuthorityOption
);
```

The regular config creation path passes `false` into that validation, while `create_config_with_transfer_hook` passes `true` and then requires a `Token-2022` config:


```
// create_config
config_parameters.validate(
    &ctx.accounts.quote_mint,
    Clock::get()?.unix_timestamp as u64,
    false,
)?;

// create_config_with_transfer_hook
config_parameters.validate(
    &ctx.accounts.quote_mint,
    Clock::get()?.unix_timestamp as u64,
    true,
)?;
```

As a result, the mint-authority variants are rejected for normal configs but are intentionally accepted for transfer-hook configs.

Pool initialization follows the same distinction. SPL token pool initialization rejects any option with mint authority. Token-2022 pool initialization rejects a non-empty mint authority only when the config is not a transfer-hook config; otherwise it passes that mint authority into `set_authority`:

```
let token_mint_authority =
    token_authority.get_mint_authority(creator.key(),
    config.fee_claimer.key());

if !config_loader.is_transfer_hook_config() {
    require!(
        token_mint_authority.is_none(),
        PoolError::InvalidTokenAuthorityOption
    );
}

anchor_spl::token_interface::set_authority(
    /* ... */
    AuthorityType::MintTokens,
    token_mint_authority,
)?;
```

This allows a Token-2022 transfer-hook pool to set the creator or partner as mint authority.

Because the comments describe the variants as unconditionally deprecated and failing, they can mislead maintainers and integrators reviewing the transfer-hook mint-authority behavior.

Recommendations:

We recommend updating the comments.

Meteora: Resolved with [@a7649a8915 ...](#)

Zenith: Verified.

[I-7] Config loader panics on short config account data

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [src/utils/config_account_loader.rs#L16-L40](#)

Description:

ConfigAccountLoader::try_from validates that the supplied account is owned by the DBC program, then immediately slices the first eight data bytes to read the account discriminator:

```
let data = account_info.try_borrow_data()?;  
let disc = &data[..8];  
let is_transfer_hook = disc == ConfigWithTransferHook::DISCRIMINATOR;
```

The loader only checks the account data length after this slice. As a result, a DBC-owned account with fewer than eight data bytes causes the slice operation to panic before the loader can return PoolError::InvalidConfigAccount.

This loader is used to support config accounts that are passed through unchecked account interfaces across multiple instruction paths. Those paths rely on the loader to gracefully reject invalid config account layouts. Short invalid config account data instead reaches an out-of-bounds slice before the discriminator and size checks complete.

Recommendations:

Meteora: Resolved with [@a7649a8915...](#)

Zenith: Verified.

[I-8] Transfer-hook handlers accept unconsumed or unused remaining accounts

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [ix_claim_partner_trading_fee.rs#L100](#)
- [ix_claim_creator_trading_fee.rs#L85](#)
- [remaining_accounts.rs#L29](#)
- [process_swap.rs#L286](#)

Description:

The transfer-hook handlers parse caller-provided remaining accounts into declared transfer-hook slices.

However, the fee-claim handlers do not require all remaining accounts to be consumed, and `claim_trading_fee2` accepts unused `TransferHookBaseReferral` slices even though it only uses `transfer_hook_base`.

The same unused-slice pattern also exists in `swap2_with_transfer_hook`: `process_swap` parses `transfer_hook_base_referral`, but only consumes it inside the referral branch when fees are charged on the base token. A caller can provide a referral hook slice when there is no referral account, or when the referral transfer is not on the base token, and the parsed account group is silently ignored.

Malformed account lists can pass silently, hiding client mistakes and making the instruction interface non-canonical.

Example Scenario:

1. A client prepares `claim_trading_fee2` with 3 valid base hook accounts and 2 trailing unrelated accounts.
2. `parse_transfer_hook_accounts` consumes the declared base slice.
3. The trailing accounts remain in `remaining_accounts`.
4. The handler never checks that `remaining_accounts` is empty.

5. The malformed transaction succeeds even though the account list does not match the instruction's expected shape.

Additional Swap Scenario:

1. A client prepares `swap2_with_transfer_hook` without a referral token account.
2. The client includes a `TransferHookBaseReferral` slice in `transfer_hook_accounts_info`.
3. `process_swap` parses the slice into `parsed_transfer_hook_accounts.transfer_hook_base_referral`.
4. Since the referral branch is skipped, those accounts are never used.
5. The malformed transaction can still succeed even though the supplied hook-account shape does not match the swap path that executed.

```
let parsed_transfer_hook_accounts =  
  parse_transfer_hook_accounts(&mut remaining_accounts,  
    &transfer_hook_accounts_info.slices)?;
```

Recommendations:

Reject unconsumed remaining accounts and unused slice types in fee-claim instructions.

```
require!(remaining_accounts.is_empty(),  
  PoolError::InvalidRemainingAccountsLength);  
require!(parsed_transfer_hook_accounts.transfer_hook_base_referral.is_none(),  
  PoolError::InvalidRemainingAccountsLength);
```

Meteora: Resolved with [@878f513c91...](#) and [@38de370917...](#). The fix partially addresses the finding. In the `swap2_with_transfer_hook` ix. The `TransferHookBaseReferral` may not be consumed if there is no referral for the swap. We will acknowledge this.

Zenith: Verified. Resolved for `claim_creator_trading_fee` and `claim_partner_trading_fee`. Acknowledged for `swap2_with_transfer_hook`.

[I-9] README omits the transfer-hook-only restriction for mint-authority options

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [README.md#L49-L49](#)

Description:

The README describes token_update_authority options 3 and 4 as allowing the creator or partner to update token metadata and mint tokens:

```
3: creator can update token metadata and mint token, 4: partner can update token metadata and mint token.
```

That description omits the PR 193 restriction that these mint-authority variants are only valid for transfer-hook configs and transfer-hook pools.

The config validation explicitly rejects mint-authority variants when the config is not being created through the transfer-hook path. [ConfigParameters::validate](#) decodes token_update_authority, then requires either is_transfer_hook or a token authority option without mint authority:

```
require!(
  is_transfer_hook || !token_authority_option.has_mint_authority(),
  PoolError::InvalidTokenAuthorityOption
);
```

The changelog documents the intended behavior: options

CreatorUpdateAndMintAuthority and PartnerUpdateAndMintAuthority are re-enabled for transfer-hook configs only, are deprecated for non-transfer-hook configs, and are rejected by the non-transfer-hook config/pool initialization paths. See [CHANGELOG.md](#).

As a result, integrators using the README as the configuration reference can incorrectly assume options 3 and 4 are valid for ordinary configs. Those configs fail at creation or initialization, while the same options are intentionally accepted only for Token-2022 transfer-hook configs.

Recommendations:

We recommend updating the README's `token_update_authority` description to state that options 3 and 4 are only valid for Token-2022 transfer-hook configs/pools, and that non-transfer-hook configs or pool initialization paths reject them.

Meteora: Resolved with [@73f7411994 ...](#)

Zenith: Verified.